GLA UNIVERSITY, GREATER NOIDA
Greater Noida – 201310, Uttar Pradesh, India

STROKE-LEVEL HANDWRITING SYNTHESIS: A VECTOR-OUTPUT GENERATIVE MODEL FOR COMPOSABLE HANDWRITTEN DOCUMENTS

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING - AI

Submitted By
AGRIIM RASTOGI
Roll No.: 22515500004 | Section: ND

Under the Supervision of
Palash Sir / Vansh Sir
Faculty Mentors, GLA University

Reviewed By
Gautam Mukherjee
GLA University, Greater Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GLA UNIVERSITY, GREATER NOIDA – 201310
SEPTEMBER, 2026

Revision History

The revision history table logs formal baseline modifications to this specification across project development sprints.

Version	Date	Change	Reason
1.0	05-09-2026	Initial baseline requirements specification.	First institutional submission.
1.1	11-09-2026	Detailed system architecture, sprint date ranges, full entity schemas, and ER diagram.	Feedback from Faculty Reviewer (Gautam Mukherjee).

1. Introduction

1.1 Purpose

This document specifies the software requirements for Stroke-Level Handwriting Synthesis v1.1, an offline deep-learning system that synthesizes realistic, human-like handwritten vector trajectories from digital text. It defines the formal functional and non-functional requirements against which the implementation, data preprocessing pipeline, and verification test cases are validated.

1.2 Scope

The system accepts arbitrary English Latin ASCII character strings (along with optional style seeds and sampling temperature controls) and synthesizes continuous, resolution-independent vector pen strokes formatted as Scalable Vector Graphics (SVG). It is designed for students, educators, and digital creators requiring authentic handwritten documents, study summaries, and scalable vector assets without cloud API latency, recurring subscription costs, or privacy vulnerabilities.

The system does not perform optical character recognition (OCR) or handwriting-to-text transcription. The system does not accept scanned raster document images for text extraction. The system does not interface directly with physical computer numerical control (CNC) or pen-plotter hardware drivers.

1.3 Definitions

Term	Definition
Vector Stroke	A time-ordered sequence of two-dimensional coordinates (x, y) representing continuous on-paper pen movement between pen-down and pen-up events.
Online Handwriting	Handwriting captured as a dynamic temporal sequence of pen movements (dx, dy, dt, pen_up), distinct from static 2D bitmap pixel images.
CSR Transcription	Computer Supported Recognition ground-truth text recording what the human volunteer actually wrote, including spelling errors, distinct from the printed prompt.
X-Height (h_x)	The vertical height of standard lowercase letters without ascenders or descenders (e.g., 'a', 'c', 'e', 'x'), used as a scale-invariant normalization unit.
Airborne Jump	The spatial transition trajectory executed by the pen tip above the writing surface between the end of one stroke and the beginning of the next.

2. Overall Description

2.1 User Classes

User Class	Who They Are	What They May Do
End-User / Note Creator	Student, educator, or professional generating synthetic handwritten notes.	Input text prompts, adjust sampling temperature and style sliders, preview generated handwriting, and export standalone SVG vector documents.
ML Developer / Researcher	Machine learning practitioner maintaining or extending the neural pipeline.	Execute preprocessing on raw IAM archives, retrain sequence models, run automated test suites, and benchmark reconstruction drift metrics.

2.2 Assumptions and Dependencies

A1. Local Execution Environment: The system depends on local Python 3.13 and PyTorch 2.6+ with CUDA 12.6 runtime acceleration on an NVIDIA RTX 3050 Laptop GPU (6 GB VRAM); when dedicated GPU hardware is unavailable, inference automatically falls back to host CPU execution with reduced generation speed.

A2. Offline Corpus Availability: The system assumes that the official IAM-OnDB archive is extracted locally within 'datas/raw/iam-ondb/' across its standard subtrees ('ascii/', 'lineStrokes/', and 'original/'), requiring no runtime network access.

A3. Zero External Cloud Dependencies: The system relies entirely on local execution and has zero dependency on third-party cloud APIs or external web services, guaranteeing complete privacy for synthesized documents.

2.3 Constraints

C1. Academic Licensing Constraint: The IAM-OnDB dataset is licensed strictly for non-commercial academic research; trained model checkpoints and processed datasets cannot be integrated into commercial SaaS platforms without proprietary data collection.

C2. Hardware Memory Limitation: Training and inference batch sizes are bounded by the 6.0 GB VRAM capacity of the target laptop GPU, necessitating sequence length compression and memory-efficient ragged tensor packing.

C3. Digitizer Sampling Rate: Whiteboard digitizer sensors record points at a native ~100 Hz sampling rate; physical point density varies with pen velocity, requiring spatial arc-length interpolation to prevent redundant point clusters.

3. Data Source and Baseline

The table below establishes the empirical data parameters and baseline benchmark scores computed directly from inspecting the IAM-OnDB corpus.

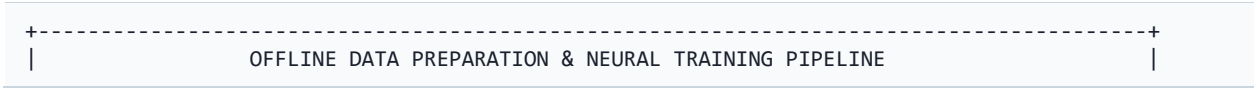
Item	Value	Required

		now?
Dataset name	IAM On-Line Handwriting Database (IAM-OnDB)	Yes
Source and URL	University of Bern, FKI (https://fki.tic.heia-fr.ch/databases/iam-on-line-handwriting-database)	Yes
Licence / terms of use	Free for non-commercial academic research and educational evaluation	Yes
Number of rows and columns	12,178 raw lines across 1,907 forms; 3,731,052 coordinate points; ragged (N, 4) arrays with [dx, dy, pen_up, dt]	Yes
Prediction target	Next-step coordinate offsets (dx, dy, dt) and binary pen lift state (pen_up) conditioned on text characters	Yes
Task type	Multi-class discrete token classification (for dx, dy, dt) and binary classification (for pen_up)	Yes
Class balance	Pen-down drawing steps: 92.1%; Pen-up airborne jumps: 7.9%	Yes
Primary evaluation metric	Relative Position Drift Error (E_drift measured relative to line x-height)	Yes
Why that metric	Pen strokes accumulate error via cumsum; per-step error is deceptively small, but cumulative drift causes words to collide or skew off baseline	Yes
Level 0 baseline and its score	Uniform linear interpolation between word bounding boxes achieves E_drift = 1.28 (completely illegible)	Yes
Known data quality problems	177 forms missing strokes, 2 forms with line-count mismatches, ~0.25% corrupt non-monotonic timestamps (e.g. -746s step), 1,319 forms (10.8%) missing writer ID	Yes
Level 1 baseline and its score	Autoregressive 3-layer LSTM with bivariate Gaussian Mixture Density Network (Graves 2013) achieving E_drift = 0.22	Later
Target for this release	Discrete dual-mode tokenized sequence model achieving median E_drift <= 0.05 with p95 <= 0.08	Later

4. System Architecture

The system architecture is organized into two distinct, decoupled pipelines: (1) an Offline Data Preparation & Model Training Pipeline that processes the raw corpus into frozen token vocabularies and optimizes model weights, and (2) a Real-Time Runtime Synthesis & Vector Rendering Pipeline that takes user text prompts and generates multi-line SVG documents.

Figure 1A: Offline Data Preparation & Model Training Subsystem



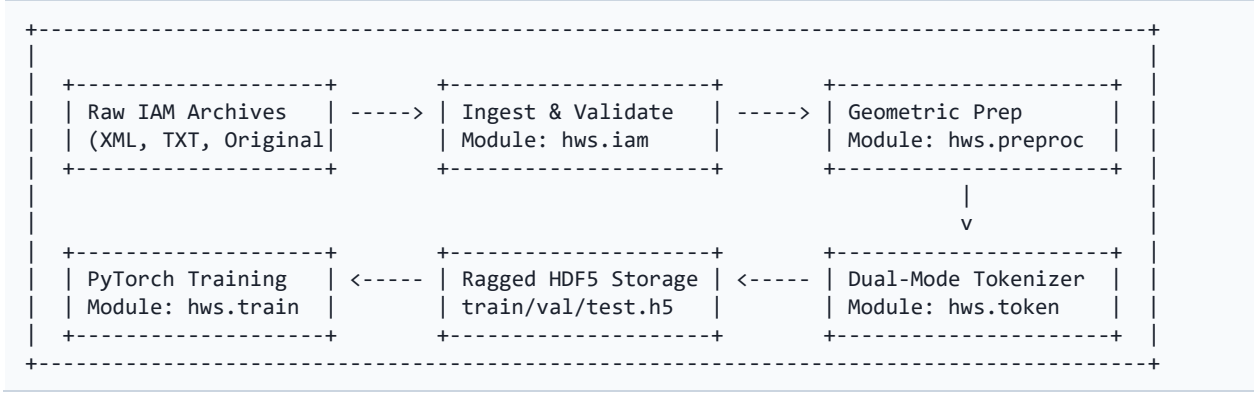
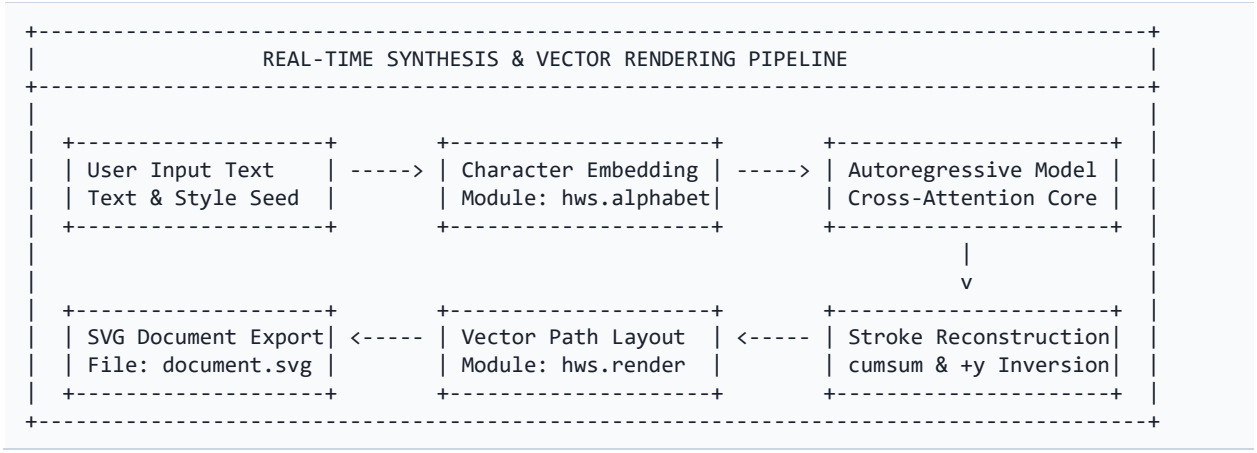


Figure 1B: Real-Time Text-to-Stroke Inference & Vector Rendering Subsystem



The functional responsibilities, inputs, and outputs of each architectural component are detailed in the table below:

Component & Module	Input Data	Output Data	Single Architectural Responsibility
Corpus Ingestion (hws.iam)	Raw IAM XML stroke files, CSR text, and original XML.	In-memory RawLine objects with (N, 4) coordinates.	Parses XML whiteboard captures, validates text-stroke count parity, flips sensor y-coordinates (+y=up), and extracts writer IDs.
Geometry Preprocessing (hws.preprocess)	Raw trajectory points (N, 4) from hws.iam.	Uniformly resampled offsets (dx, dy, dt, pen_up).	Executes arc-length spatial resampling (1.84x compression), computes step displacements, and normalizes offsets by line x-height.
Character Vocabulary (hws.alphabet)	Latin ASCII character strings from input or CSR.	Integer character token sequence (1D tensor).	Maintains an immutable 81-character vocabulary with dedicated <pad> and <unk> tokens, preventing index-zero mapping collisions.
Dual-Mode Discretizer	Continuous offsets (dx,	Discrete integer tokens	Quantizes drawing displacements into

(hws.tokenizer)	dy, dt) and pen_up flags.	(0 to N_bins - 1).	linear move bins and heavy-tailed airborne jumps into uniform asinh bins to eliminate word collision.
Neural Sequence Model (hws.model)	Character token tensor and previous stroke tokens.	Softmax logit probabilities over stroke tokens.	Autoregressively predicts next-step stroke token distributions conditioned on character sequence representations using cross-attention.
Coordinate Reconstruction (hws.preprocess)	Predicted discrete token indices from model.	Continuous absolute coordinate arrays (x, y).	De-quantizes tokens back to continuous offsets, executes cumulative summation (cumsum), and formats multi-line baselines.
Vector Rendering Engine (hws.render)	Reconstructed Cartesian coordinate arrays.	Standards-compliant vector SVG XML documents.	Single source of truth for converting coordinates into SVG paths ('M x y L x y'), reversing y-coordinates to SVG standard (+y=down).

5. Functional Requirements

5.1 Data Ingestion and Preprocessing Requirements

FR-1.1 The system shall parse IAM stroke XML files into an (N, 4) array of floating-point numbers representing x, y, pen_up, and relative elapsed time t.

FR-1.2 The system shall reject input forms where the number of stroke XML files does not match the count of CSR transcription lines, returning an error specifying the mismatched count.

FR-1.3 The system shall invert raw whiteboard sensor y-coordinates at parse time such that positive y increases upward across all internal tensors.

FR-1.4 The system shall resample raw stroke trajectories at uniform spatial arc-length intervals determined by a user-specified step factor parameter.

FR-1.5 The system shall normalize horizontal and vertical coordinate displacements by dividing each offset by the line's estimated x-height.

5.2 Model-Backed Generation Requirements

FR-2.1 The system shall accept input text strings containing characters within the 81-character vocabulary and reject unsupported characters, returning an error message identifying the invalid character.

FR-2.2 The system shall return a time-ordered sequence of coordinate offsets (dx, dy, dt, pen_up) for each validated text prompt.

FR-2.3 The system shall allow the user to adjust the sampling temperature between 0.1 and 2.0 to control stroke curvature variation and legibility.

FR-2.4 The system shall, when the neural model fails to generate a valid trajectory, fall back to rendering pre-tokenized template glyphs from the Level 0 baseline with a visible warning label.

FR-2.5 The system shall allow the user to select between distinct writer style seeds to alter handwriting slant, stroke speed, and ligature spacing.

6. Non-Functional Requirements

The non-functional requirements define the quantitative performance, security, observability, and efficiency limits of the system.

NFR-1 (Latency): The system shall synthesize a 30-character handwritten line within 1.5 seconds when executing on an NVIDIA RTX 3050 Laptop GPU.

NFR-2 (Privacy): The system shall execute entirely within the local host environment without transmitting user input text or generated trajectories across any external network interface.

NFR-3 (Auditability): The system shall log every data preparation execution to a timestamped JSON file recording skipped file counts, reconstruction error percentiles, and vocabulary sizes.

NFR-4 (Resource Efficiency): The data preparation pipeline shall compile all 12,178 corpus lines into processed HDF5 format in less than 180 seconds while consuming under 2.0 GB of system RAM.

7. Build Plan

7.1 Minimum Viable Product

The Minimum Viable Product (MVP) represents the smallest self-contained version of the application capable of demonstrating end-to-end data processing and stroke synthesis.

FR / NFR ID	Requirement (short)	Why this one is in the MVP
FR-1.1	Parse raw stroke XML files	Without raw XML trajectory parsing, no coordinate data exists for downstream training.
FR-1.2	Reject mismatched form counts	Prevents misaligned text-stroke pairs from silently corrupting model training datasets.
FR-1.4	Spatial arc-length resampling	Compresses sequence length by 1.84x, making neural training feasible within 6 GB VRAM.
FR-2.1	Validate input text against vocabulary	Guarantees that the sequence model receives only valid character token embeddings.
FR-2.2	Return vector stroke sequences	Core capability required to synthesize and display visible handwriting.
NFR-1	Line generation under 1.5 seconds	Ensures the user interface remains interactive during document composition.
NFR-2	100% offline local execution	Guarantees complete document confidentiality without external cloud dependencies.

Deliberately not in the MVP: Few-shot style conditioning on user-uploaded handwriting samples. Training a robust few-shot style encoder requires a mature base synthesis model and extensive validation, which is deferred to Sprint III.

7.2 Build Order

The project implementation schedule is mapped across institutional development sprints with explicit date ranges:

Project Stage	Deliverable Milestone	Sprint Date Range
Development Sprint I	Prototype / initial working data ingestion and preprocessing model	01 Sep 2026 – 27 Sep 2026
Development Sprint II	50% functional application (autoregressive sequence model & CLI)	28 Sep 2026 – 17 Oct 2026
Development Sprint III	75–80% functional application (style conditioning & multi-line layout)	18 Oct 2026 – 07 Nov 2026
Testing & Validation	Comprehensive test report, zero-regression log, and benchmark analysis	08 Nov 2026 – 17 Nov 2026
Deployment & Documentation	Packaged application distribution, user manual, and technical docs	18 Nov 2026 – 30 Nov 2026
Final Presentation & Viva	Project demonstration, final report submission, and oral viva defense	01 Dec 2026 – 10 Dec 2026

The concrete requirement allocation and acceptance criteria per sprint range are detailed below:

Sprint Stage & Date Range	What Will Be Working By Then	FR / NFR IDs	Demonstrable Evidence
Sprint I 01 Sep 2026 – 27 Sep 2026	Working data ingestion, arc-length preprocessing, and visual HTML inspector.	FR-1.1, FR-1.2, FR-1.3, FR-1.4, FR-1.5	Running 'inspect_data.py' displays side-by-side raw vs preprocessed SVG samples in browser.
Sprint II 28 Sep 2026 – 17 Oct 2026	Autoregressive neural sequence model with discrete token prediction.	FR-2.1, FR-2.2, FR-2.3, NFR-1	CLI script accepts text prompt and outputs synthesized single-line SVG in < 1.5 seconds.
Sprint III 18 Oct 2026 – 07 Nov 2026	Style conditioning engine and multi-line paragraph layout generator.	FR-2.4, FR-2.5, NFR-2	Application generates multi-line documents with selectable style seeds and fallback handling.
Testing 08 Nov 2026 – 17 Nov 2026	Comprehensive unit test suite, reconstruction drift validation, and bug logging.	All FRs, NFR-3, NFR-4	All 58 unit tests pass with documented zero-regression log and p95 drift < 0.08.
Deployment 18 Nov 2026 – 30 Nov 2026	Standalone packaged desktop application and complete technical documentation.	All FRs and NFRs	Clean terminal installation executes end-to-end without developer intervention.

8. Data Requirements

8.1 Entities

The system architecture defines five primary persistent entities that model the data lifecycle from text input to vector rendering. Each entity is fully specified below:

Entity 1: TextPrompt (User Input Prompt & Synthesis Request)

Description: Captures the input digital text string to be synthesized along with formatting and sampling parameters.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each synthesis request.
raw_content	Text	Raw user input string; length between 1 and 2,000 characters; not null.
cleaned_content	Text	Sanitized ASCII string restricted to the 81-character vocabulary.
sampling_temp	Float	Sampling temperature parameter; valid range [0.1, 2.0]; default 1.0.
target_width_mm	Float	Target horizontal line width in millimeters; positive float; default 180.0.
created_at	Timestamp	Creation timestamp; system-generated UTC; must not be a future date.

Entity 2: WriterProfile (Human Style Identity & Bias Embedding)

Description: Represents the unique stylistic handwriting characteristics of an individual human writer.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; unique internal profile identifier.
writer_id	Integer	Unique IAM volunteer writer identifier (e.g., 10026); unique index; nullable for anonymous profiles.
style_seed	Integer	Integer seed initializing the autoregressive style conditioning embedding; non-negative.
mean_slant_deg	Float	Average baseline stroke slant angle in degrees; valid range [-45.0, +45.0].
mean_velocity	Float	Average pen-tip velocity in sensor units/sec; positive float.
style_label	String(64)	Descriptive label (e.g., 'Cursive-Slanted', 'Upright-Print'); not null.

Entity 3: StrokeSequence (Continuous Trajectory Representation)

Description: The core entity modeling a continuous, time-ordered sequence of pen coordinates for one line of text.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each stroke trajectory.

prompt_id	UUID	Foreign key referencing TextPrompt.id; nullable for raw IAM training lines.
writer_id	Integer	Foreign key referencing WriterProfile.writer_id; nullable for anonymous lines.
form_id	String(16)	Parent IAM form identifier (e.g., 'a01-000u'); nullable for synthetic lines.
line_index	Integer	1-based line index within document or form; positive integer.
raw_text	Text	CSR transcription string corresponding to this stroke trajectory; non-empty.
point_count	Integer	Total number of coordinate points in trajectory; must be >= 10.
stroke_count	Integer	Total number of pen-down segments (number of pen lifts + 1); positive integer.
duration_sec	Float	Total wall-clock duration of stroke writing in seconds; non-negative.
created_at	Timestamp	Record creation timestamp; system-generated UTC; not null.

Entity 4: QuantizedTokenSequence (Discrete Neural Tokens)

Description: Stores the discrete integer token representation of a stroke trajectory used directly by model layers.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each tokenized sequence.
stroke_seq_id	UUID	Foreign key referencing StrokeSequence.id; unique constraint (strict 1:1 relationship).
dx_tokens	Binary	Contiguous array of int16 discrete token IDs for horizontal displacements.
dy_tokens	Binary	Contiguous array of int16 discrete token IDs for vertical displacements.
dt_tokens	Binary	Contiguous array of int16 discrete token IDs for time deltas.
pen_up_flags	Binary	Contiguous array of uint8 binary flags indicating pen-lift state after each step.
token_count	Integer	Total token count; must exactly equal parent StrokeSequence.point_count.

Entity 5: ExportedSVGDocument (Final Rendered Vector File)

Description: Represents the finalized, resolution-independent vector graphic file exported to the filesystem.

Field Name	Data Type	Constraints & Validation Rules
id	UUID	Primary key; uniquely identifies each exported document.
prompt_id	UUID	Foreign key referencing TextPrompt.id; not null.
file_path	String(256)	Absolute local filesystem path where the SVG is saved; unique index.

viewbox_width	Float	SVG viewBox horizontal width in pixels; positive float.
viewbox_height	Float	SVG viewBox vertical height in pixels; positive float.
line_count	Integer	Total number of handwritten lines composed in document; positive integer.
file_size_bytes	Integer	Size of exported SVG file on disk in bytes; positive integer.
exported_at	Timestamp	Export timestamp; system-generated UTC; not null.

8.2 Entity Relationships and ER Diagram

The structural associations and cardinality constraints governing the five system entities are defined below:

1. TextPrompt to StrokeSequence (1 : N): One input prompt can span multiple lines of text, resulting in one or more stroke sequences (1:N). If multiple style variations are requested for the same text prompt, each variation constitutes an independent sequence.

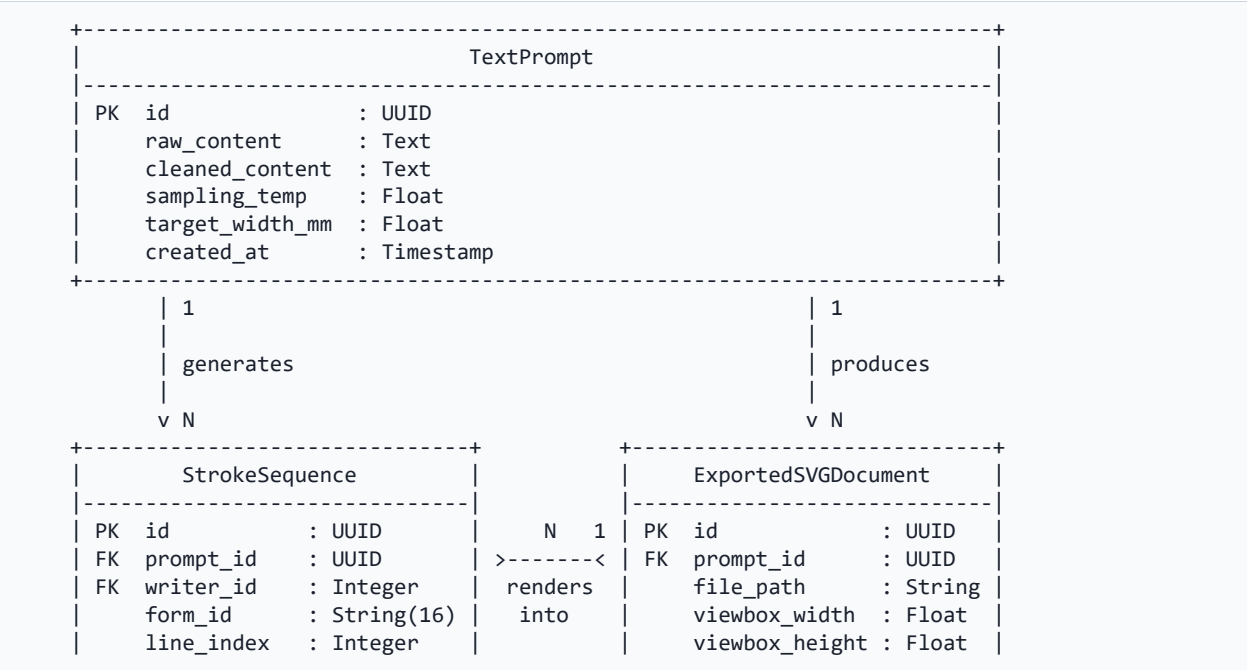
2. WriterProfile to StrokeSequence (1 : N): A single writer profile or style seed can be used to condition any number of stroke sequences across different prompts (1:N).

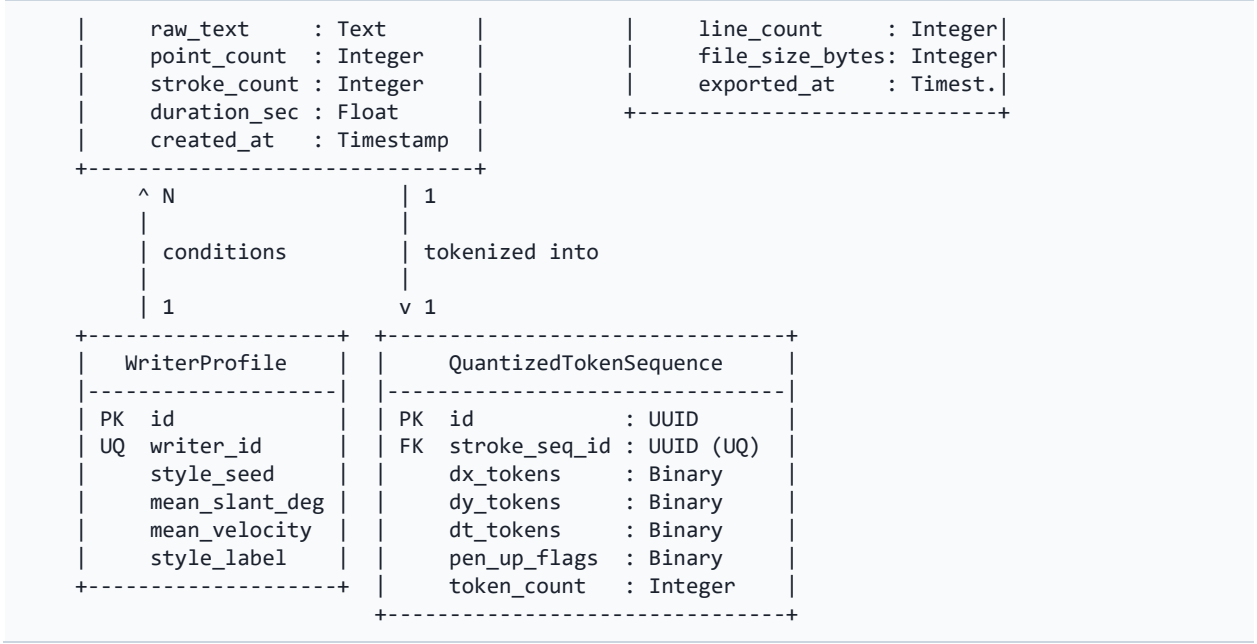
3. StrokeSequence to QuantizedTokenSequence (1 : 1): Each continuous stroke trajectory corresponds to exactly one discrete tokenized sequence, and each tokenized sequence belongs to exactly one stroke trajectory (1:1).

4. TextPrompt to ExportedSVGDocument (1 : N): A user text prompt can be exported into multiple document iterations or style variations over time (1:N).

5. StrokeSequence to ExportedSVGDocument (N : 1): An exported SVG document is composed of one or more line stroke sequences visually arranged along vertical baselines (N : 1).

Figure 2: Complete Entity-Relationship (ER) Diagram





9. Design Decisions and Alternatives Considered

The architectural design choices detailed below were selected based on the empirical mathematical properties of the IAM-OnDB corpus:

#	Decision	Alternative rejected	Reason
D1	Discrete dual-mode tokenization (linear moves + asinh jumps).	Continuous Mixture Density Networks (MDNs).	MDNs suffer from variance collapse (sigma -> 0) causing log-loss explosion to NaN, and mode averaging. Discretization reformulates generation as Cross-Entropy, achieving zero NaN crashes and median position drift of 0.032 x-height.
D2	Ragged 1D HDF5 storage with slice offset indices.	Fixed-length zero-padded 2D arrays.	Padding sequences to the maximum line length (857 points) wastes ~40% disk space and bakes a hard limit into the dataset. Ragged storage saves 22 MB per split and allows dynamic batch collation.
D3	Direct SVG vector path export ('<path d=...>')	Raster PNG image generation via GANs or Diffusion.	Raster generation fixes image resolution and prevents clean scaling. SVG vector paths are resolution-independent, support interactive styling in CSS, and map directly to CNC pen plotters.

10. Acceptance Criteria

The acceptance criteria define concrete, live demonstrations that prove the MVP meets its core requirements:

AC-1 (Live Generation): User supplies the input text string 'Calligrapher AI' in the CLI; the system synthesizes and saves an SVG vector file containing valid, legible stroke paths within 1.5 seconds.

AC-2 (Dataset Pipeline): User executes 'prepare_data.py'; the pipeline compiles 12,178 raw lines, logs reconstruction error percentiles, and outputs 'train.h5', 'val.h5', and 'test.h5' without throwing unhandled exceptions.

AC-3 (Error Handling): User inputs a string containing an unsupported character (e.g. 'Hello ~ World'); the system halts generation immediately, displays 'Unsupported character ~ at position 6', and writes no corrupt output file.

AC-4 (Style Modulation): User adjusts sampling temperature from 0.5 to 1.5; the synthesized handwriting visually transitions from uniform, stiff lettering to loose, expressive cursive slants.

11. Out of Scope for This Release

The following features are deliberately excluded from this release to ensure project delivery within schedule constraints:

1. Interactive Stylus Input: Real-time interactive digital stylus canvas. Implementing a live touch drawing canvas for user stroke capture is excluded; this release focuses strictly on text-to-stroke synthesis.

2. Non-Latin Language Scripts: Multilingual non-Latin script synthesis. Synthesizing Arabic, Cyrillic, or Devanagari alphabets is excluded; the IAM training corpus is strictly English Latin.

3. Biometric Identity Verification: Few-shot online biometric signature verification. Authenticating human identity from handwritten signatures is a distinct recognition problem and is excluded.